

ОТЧЁТ
по практической работе
«Клиентские атаки: XSS и CSRF»

Выполнил: Аньшин Сергей

2026

Цель работы и используемая среда

Изучить клиентские уязвимости XSS и CSRF на примере учебного приложения заметок; установить причины небезопасного поведения и рассмотреть исправления и контрольные проверки.

Среда, указанная в исходном отчёте: Kali Linux, Firefox, PHP и MySQL. Приложение доступно по адресу `http://localhost/lesson4/`. Исходные файлы расположены в `/var/www/html/lesson4/`; для редактирования использовался GNU nano.

Порядок разделов соответствует оглавлению методички «4. Клиентские атаки»: обзор XSS, хранимая XSS, отражаемая XSS, защита от XSS; затем обзор, эксплуатация и защита от CSRF. Изображения взяты из исходного отчёта. Для опытов без представленных результатов указан статус проверки.

1. XSS (Cross-Site Scripting)

1.1. Обзор уязвимости XSS

XSS возникает, когда недоверенные данные попадают в страницу и интерпретируются браузером как исполняемый код. Отражаемая XSS связана с данными текущего запроса, хранимая - с ранее сохранённым содержимым. При DOM-based XSS опасная обработка данных происходит в клиентском коде.

В приложении проверяются заголовок заметки из параметра `title` и содержимое заметки. Безопасная учебная нагрузка `alert(1)` позволяет увидеть факт выполнения JavaScript без обращения к внешним ресурсам.

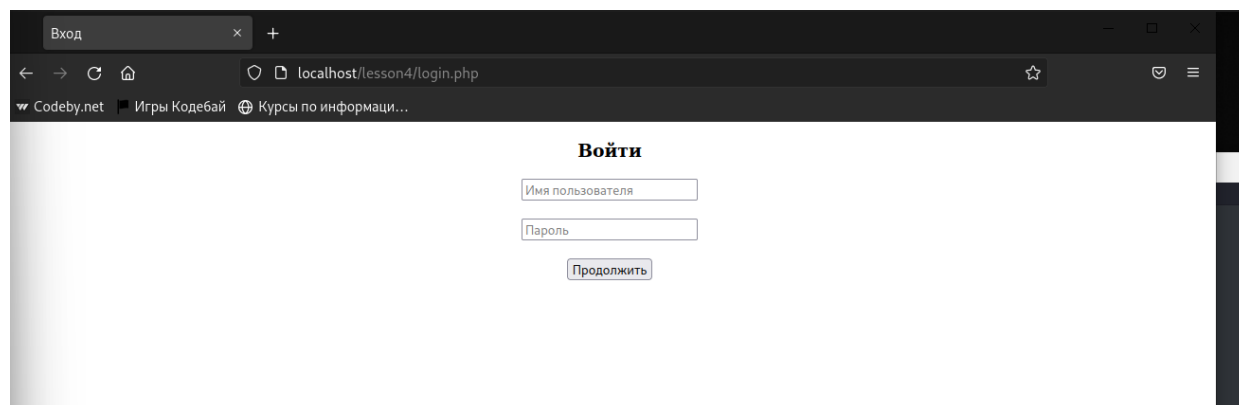


Рисунок 1. Страница входа в учебное приложение

1.2. Эксплуатация хранимой XSS

После авторизации приложение позволяет создать заметку с заголовком и содержимым, сохранить её и открыть отдельной страницей. Эти действия образуют путь, на котором возможна хранимая XSS.

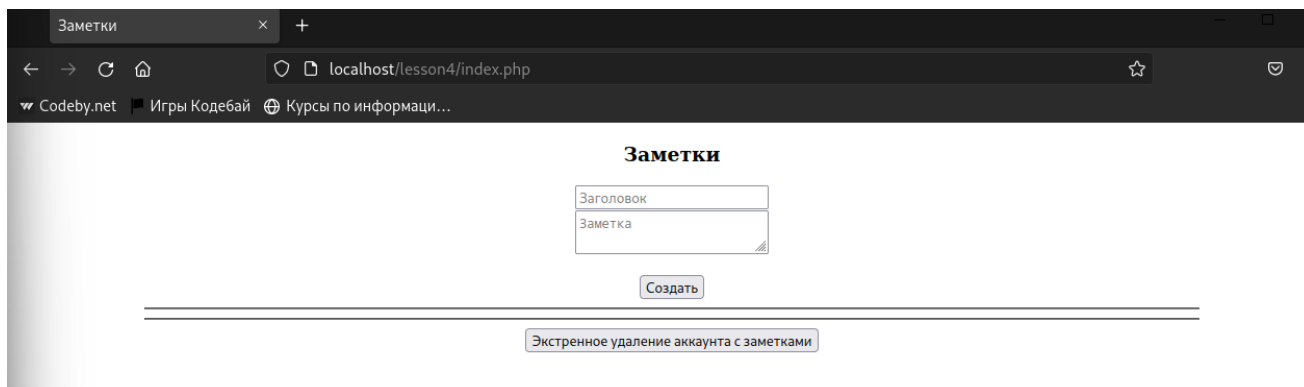


Рисунок 2. Форма создания заметки

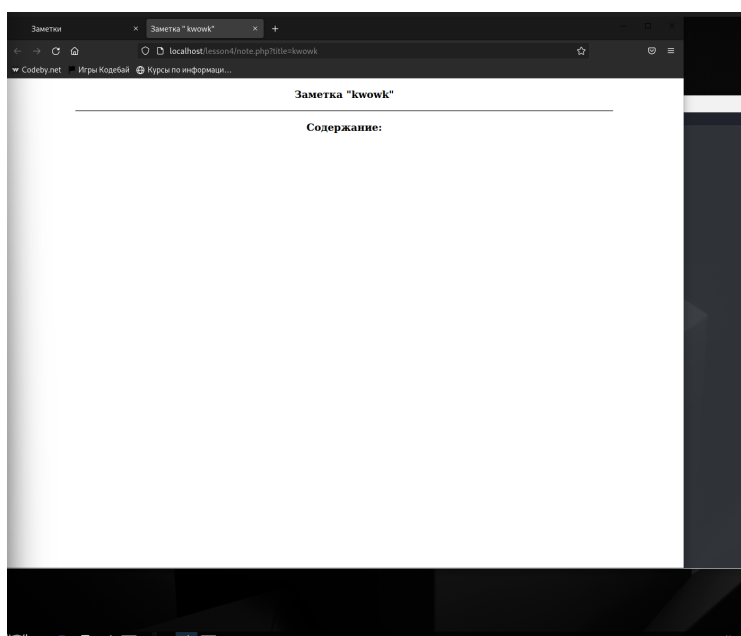


Рисунок 3. Открытая тестовая заметка

Для проверки в содержимое отдельной тестовой заметки вводится конструкция `<script>alert(1)</script>`. Затем заметка сохраняется и повторно открывается. Выполнение сценария при загрузке сохранённого содержимого указывает на хранимую XSS.

Статус: в исходном отчёте показана работа с заметками, но отдельного результата выполнения сохранённой нагрузки нет. Поэтому эксплуатация хранимой XSS в этой работе не считается подтверждённой. Методический сценарий приведён на стр. 4-11 методички.

1.3. Эксплуатация отражаемой XSS

В параметр title страницы note.php передана тестовая конструкция, вызывающая alert(1). В исходном отчёте зафиксировано появление диалогового окна. Это демонстрирует выполнение внедрённого JavaScript в браузере.

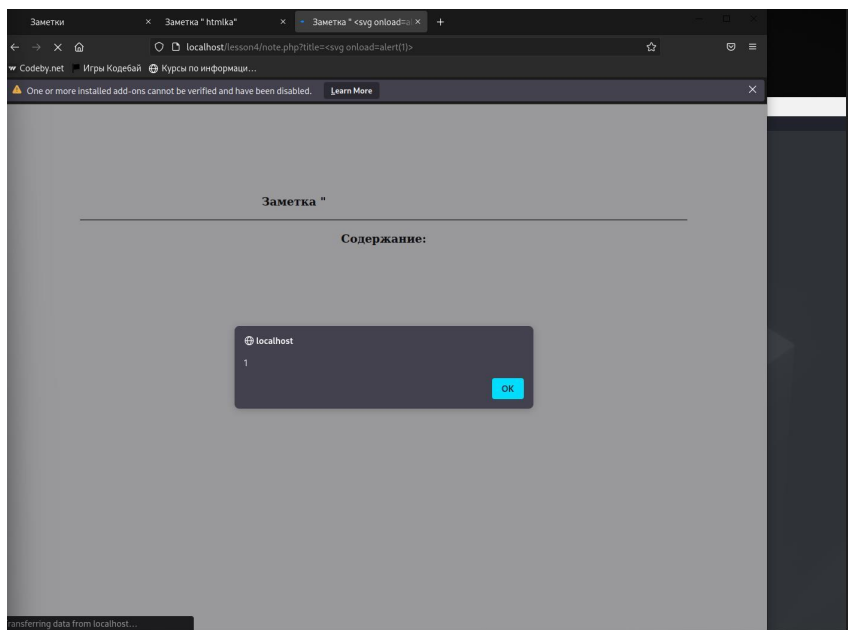


Рисунок 4. Выполнение тестовой нагрузки: окно alert(1)

Для поиска причины выполнен поиск обращений к title в файле note.php. Согласно исходному отчёту, параметр выводился в тегах title и h3 без экранирования.

```
grep -n "title" /var/www/html/lesson4/note.php
```

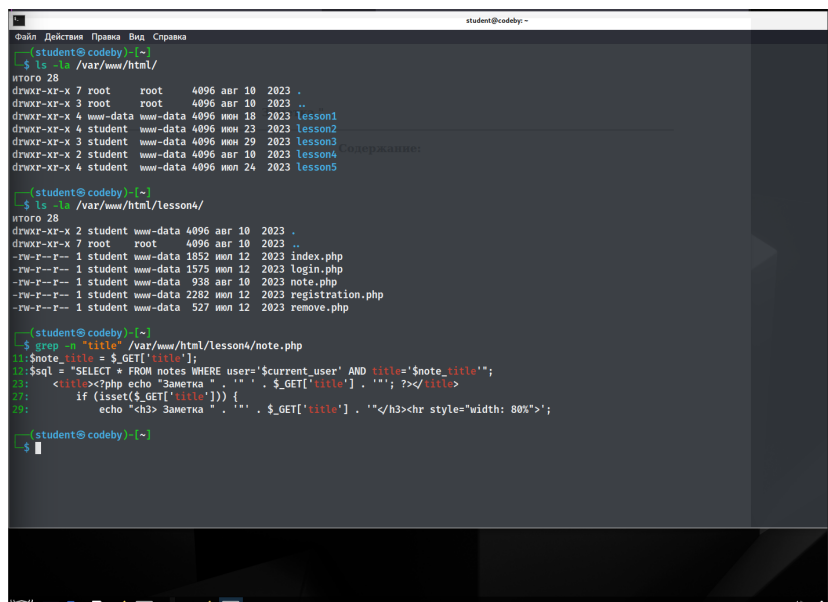


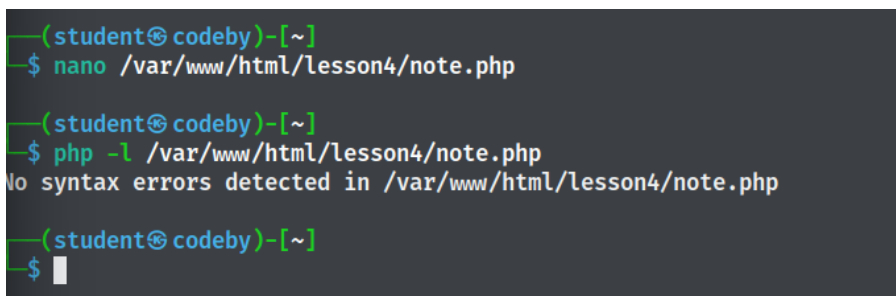
Рисунок 5. Поиск мест вывода параметра title в note.php

1.4. Защита от XSS

В исходном отчёте для вывода заголовка применена функция htmlspecialchars() с флагом ENT_QUOTES и кодировкой UTF-8. В текстовом HTML-контексте она заменяет специальные символы сущностями, поэтому пользовательский ввод не становится разметкой.

```
htmlspecialchars($_GET['title'], ENT_QUOTES, 'UTF-8')
```

После изменения проверен синтаксис файла. Успешная проверка php -l означает отсутствие синтаксических ошибок, но не заменяет повторное испытание в браузере.



```
(student@codeby)~[~]
$ nano /var/www/html/lesson4/note.php

(student@codeby)~[~]
$ php -l /var/www/html/lesson4/note.php
No syntax errors detected in /var/www/html/lesson4/note.php

(student@codeby)~[~]
$
```

Рисунок 6. Проверка синтаксиса note.php

При повторном открытии тестовой страницы конструкция отображается как текст, без выполнения сценария.

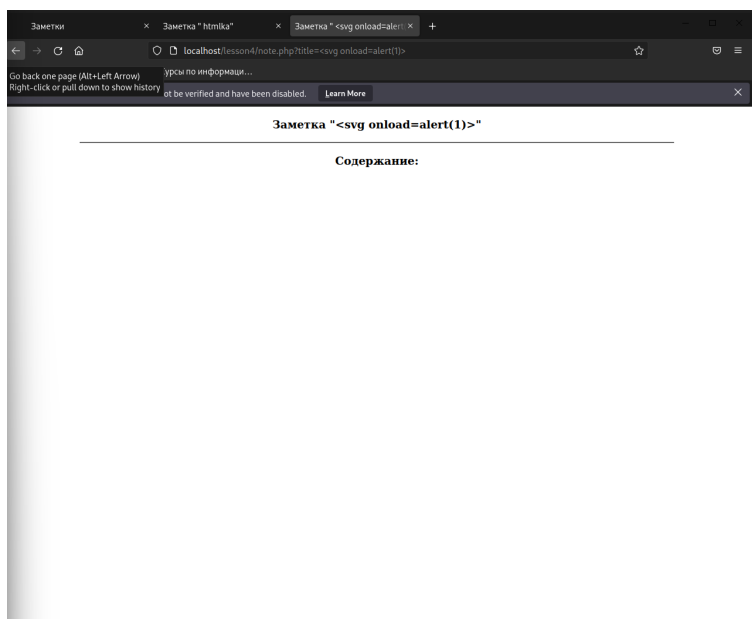


Рисунок 7. Отображение нагрузки после экранирования

Область подтверждённого результата ограничена показанным выводом заголовка. Для защиты содержимого заметки требуется экранировать его при выводе и повторить опыт из п. 1.2. Заголовок нужно обработать во всех местах использования. Для контекстов JavaScript, CSS и URL требуются соответствующие им способы защиты.

2. CSRF (Cross-Site Request Forgery)

2.1. Обзор уязвимости CSRF

CSRF - выполнение нежелательного действия от имени авторизованного пользователя через запрос, который инициирован без его намерения. Браузер может автоматически добавить cookie, а сервер ошибочно принять запрос за разрешённое действие. Возможность атаки зависит также от способа отправки запроса и настроек cookie.

2.2. Эксплуатация CSRF

В учебном приложении функция «Экстренное удаление аккаунта с заметками» была реализована ссылкой на `remove.php`. Таким образом, переход по ссылке инициировал операцию изменения данных через GET.

```
<a href="/lesson4/remove.php">  
    Экстренное удаление аккаунта с заметками  
</a>
```

По описанию исходного отчёта, `remove.php` определял пользователя через cookie `User` и выполнял удаление без проверки метода и CSRF-токена. После перехода браузер открывал страницу входа.

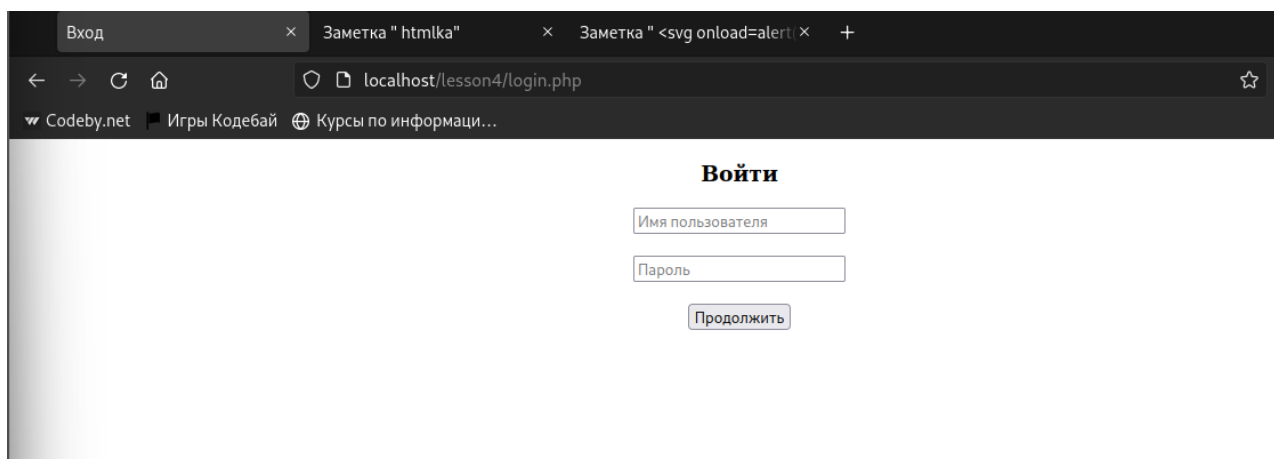


Рисунок 8. Страница входа после перехода на обработчик удаления

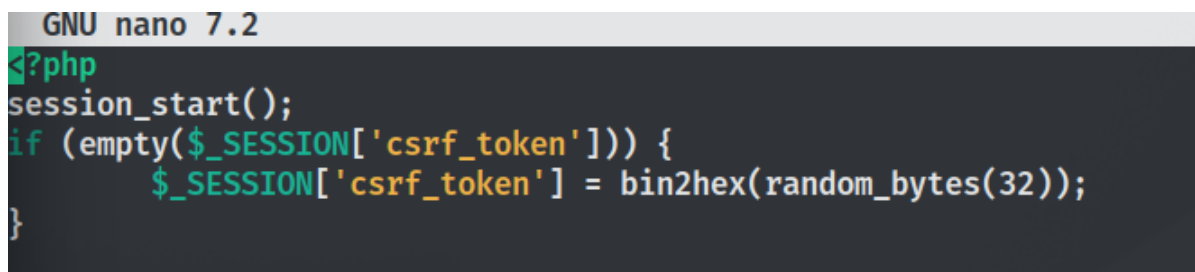
Это иллюстрирует небезопасный способ запуска операции. Для полного подтверждения результата удаления дополнительно проверяются записи аккаунта и заметок либо невозможность повторного входа. Одного перенаправления недостаточно, чтобы доказать изменение базы данных.

Доверие изменяемому значению cookie `User` без проверки серверной сессии является отдельной проблемой авторизации. Добавление CSRF-токена само по себе её не устраняет.

2.3. Защита от CSRF

В исходном отчёте описана защита из двух частей: форма POST с токеном сессии и проверка метода и токена на сервере до удаления данных.

Создание токена и формы. В index.php запускается сессия и создаётся случайный токен. Он включается в скрытое поле формы.



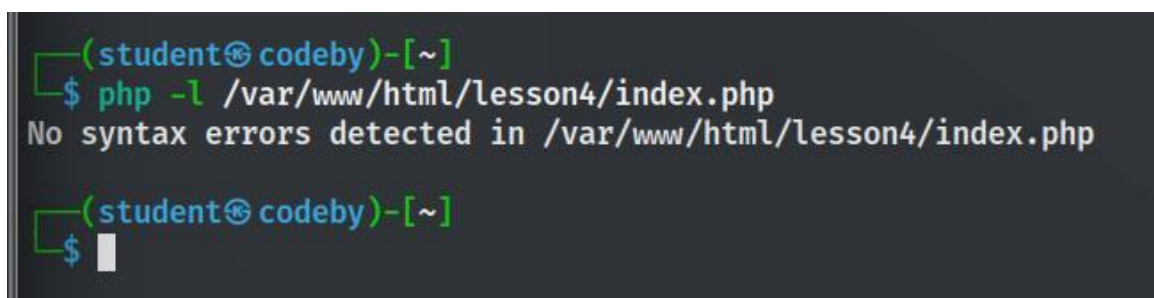
```
GNU nano 7.2
<?php
session_start();
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

Рисунок 9. Генерация CSRF-токена в index.php

```
session_start();
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

<form action="/lesson4/remove.php" method="POST">
  <input type="hidden" name="csrf_token"
    value="<?= htmlspecialchars($_SESSION['csrf_token'],
      ENT_QUOTES, 'UTF-8') ?>">
  <button type="submit">Удалить аккаунт с заметками</button>
</form>
```

session_start() выполняется до вывода HTML. Токен хранится на сервере в текущей сессии; при выводе в атрибут формы его значение экранируется. POST без проверки токена не обеспечивает защиту от CSRF.



```
(student@codeby)-[~]
$ php -l /var/www/html/lesson4/index.php
No syntax errors detected in /var/www/html/lesson4/index.php

(student@codeby)-[~]
$
```

Рисунок 10. Проверка синтаксиса index.php

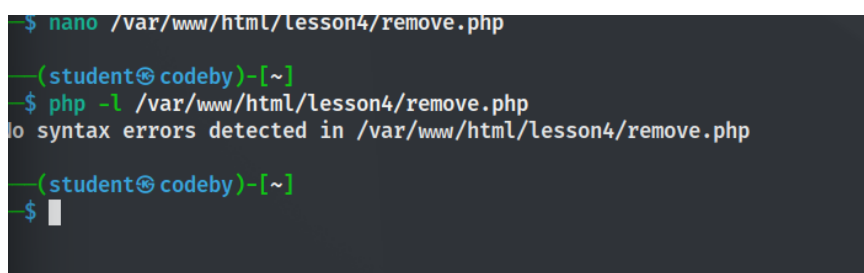
2.3. Защита от CSRF (продолжение)

Проверка в обработчике. В `remove.php` добавлены запрет GET-запросов и сравнение токена формы с токеном текущей сессии через `hash_equals()`. Эти проверки должны предшествовать DELETE-запросам.



```
student@codeby: ~  
Файл Действия Правка Вид Справка  
GNU nano 7.2 /var/www/html/lesson4/remove.php *  
<?php  
session_start();  
  
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {  
    http_response_code(405);  
    exit('Method not allowed');  
}  
  
if (empty($_POST['csrf_token']) ||  
    empty($_SESSION['csrf_token']) ||  
    !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {  
    http_response_code(403);  
    exit('Invalid CSRF token');  
}  
  
if (!isset($_COOKIE['User'])) {  
    header("Location: /lesson4/login.php");  
}
```

Рисунок 11. Проверки HTTP-метода и CSRF-токена в `remove.php`



```
$ nano /var/www/html/lesson4/remove.php  
- (student@codeby)-[~]  
$ php -l /var/www/html/lesson4/remove.php  
no syntax errors detected in /var/www/html/lesson4/remove.php  
- (student@codeby)-[~]  
$
```

Рисунок 12. Проверка синтаксиса `remove.php`

Для устойчивой обработки некорректного ввода рекомендуется явно проверять тип токена. Это предотвращает ошибку при передаче `csrf_token` как массива. Пример усиленного блока проверки:

```
$expected = $_SESSION['csrf_token'] ?? null;  
$received = $_POST['csrf_token'] ?? null;  
if (!is_string($expected) || $expected === '' ||  
    !is_string($received) || $received === '' ||  
    !hash_equals($expected, $received)) {  
    http_response_code(403);  
    exit('Invalid CSRF token');  
}
```

При отклонении GET-запроса используется HTTP 405; дополнительно указывается заголовок `Allow: POST`. После проверок токена необходимы проверка авторизации и прав, а затем согласованное удаление аккаунта и связанных заметок.

2.3. Защита от CSRF (контрольные проверки)

При прямом открытии remove.php браузер отправляет GET. В представленном результате обработчик выводит сообщение Method not allowed.

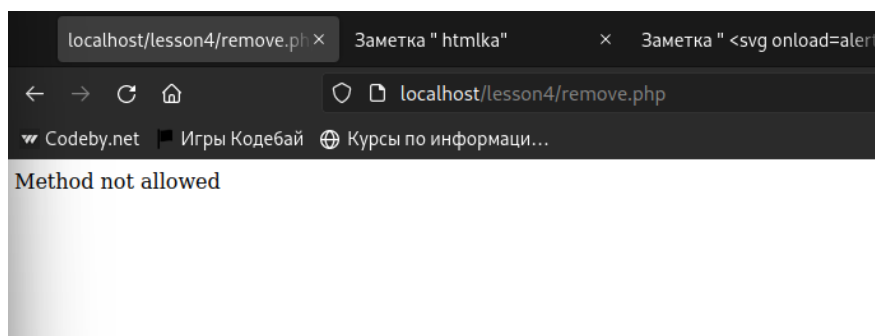


Рисунок 13. Отклонение прямого GET-запроса

При отправке формы с корректным токеном в исходном отчёте описано выполнение удаления и переход на страницу авторизации.

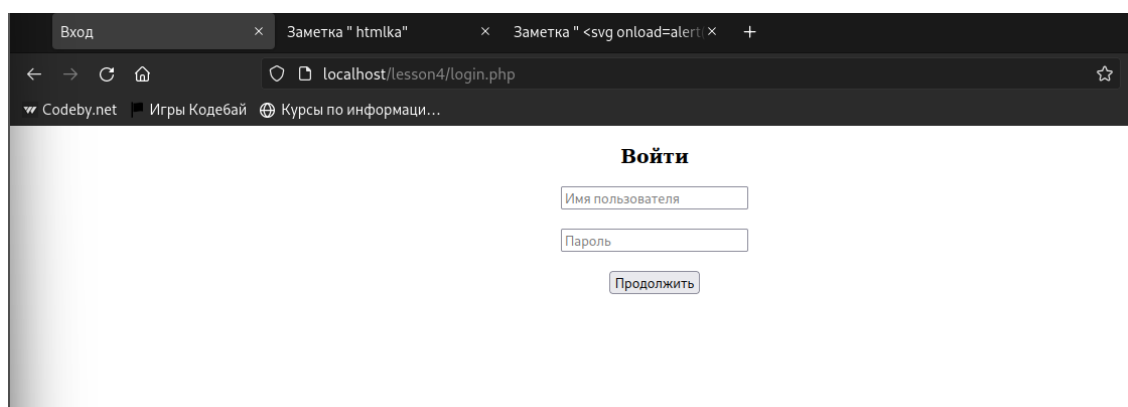


Рисунок 14. Страница входа после отправки формы удаления

Для завершения проверки:

1. Во вкладке Network подтвердить HTTP 405 для GET и проверить, что аккаунт сохранён.
2. Отправить POST без токена и с неверным токеном: ожидается HTTP 403 без удаления.
3. Передать токен другой тестовой сессии: ожидается HTTP 403.
4. Отправить корректный POST: проверить фактическое удаление тестового аккаунта и его заметок.

Результаты отрицательных проверок токена в исходных материалах отсутствуют; указанные коды и сохранность данных являются ожидаемыми результатами, а не новыми результатами испытаний.

Результаты и вывод

В работе рассмотрено учебное приложение заметок. В исходных материалах отражены выполнение тестовой нагрузки через title, экранирование вывода заголовка и повторное отображение нагрузки как текста. Причина отражаемой XSS - включение недоверенных данных в HTML без контекстного экранирования.

Для операции удаления аккаунта описан переход от ссылки GET к форме POST с CSRF-токеном. Показаны генерация токена, серверные проверки и отклонение прямого перехода. Основу защиты составляет проверка непредсказуемого токена, связанного с текущей сессией, до изменения данных.

Хранимая XSS рассмотрена в отдельном пункте согласно порядку методички. Для подтверждения её проверки необходим опыт с сохранением и повторным открытием нагрузки в содержимом заметки. Полный вывод об исправлении CSRF требует отрицательных проверок токена и проверки состояния аккаунта и заметок.

Таким образом, представленные материалы подтверждают отдельные этапы исследования и иллюстрируют выбранные способы защиты. Полнота защиты всего приложения не установлена. Практический результат работы - разбор небезопасного вывода и удаления данных, их исправлений и критериев повторной проверки.

Использованные материалы

1. «4. Клиентские атаки.pdf», 33 страницы. Разделы 1.1-1.4 и 2.1-2.3; порядок основных пунктов сохранён в настоящем отчёте.
2. «Отчет_Клиентские_атаки_XSS_CSRF (1).pdf» - исходный текст и 14 изображений практической работы.